Implementazione dell'algoritmo del Simplesso attraverso programmazione parallela con CUDA

Corso
Ricerca Operativa
Anno Accademico
2025/2026
Studente
Alessandro Porcheddu

Indice

1 **Introduzione** **3**

2 **Fondamenti Teorici** **4**

2.1 Il Metodo del Simplesso in Formato Tableau 4

2.2 Metodo delle Due Fasi 4

3 **Analisi delle Sfide** **6**

3.1 Sfide di Parallelizzazione 6

3.2 Stabilità Numerica 6

4 **Progettazione CUDA** **8**

4.1 Architettura del Sistema 8

4.2 Struttura Dati 9

4.3 Kernel CUDA 9

5 **Risultati Sperimentali** **10**

5.1 Dataset e Metodologia 10

5.2 Prestazioni CUDA su Dati-LP 10

5.3 Confronto Solver CUDA e HiGHS su Netlib 11

5.4 Errore Numerico su Netlib 12

6 **Conclusioni** **13**

1 Introduzione

La programmazione lineare rappresenta una delle aree fondamentali dell'ottimizzazione matematica, con applicazioni che spaziano dalla logistica alla finanza, dalla produzione industriale all'allocazione di risorse. Un problema di programmazione lineare in forma standard si esprime come:

$$\min \mathbf{c}^T \mathbf{x} \quad \text{s.t.} \quad A\mathbf{x} = \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0}$$

dove $\mathbf{x} \in \mathbb{R}^n$ rappresenta il vettore delle variabili decisionali, $\mathbf{c} \in \mathbb{R}^n$ i coefficienti della funzione obiettivo, $A \in \mathbb{R}^{m \times n}$ la matrice dei vincoli e $\mathbf{b} \in \mathbb{R}^m$ il termine noto.

Il metodo del Simplesso, proposto da George Dantzig nel 1947, rimane tuttora uno degli algoritmi più utilizzati per la risoluzione di problemi LP di medie dimensioni. Nonostante la sua complessità teorica esponenziale nel caso peggiore, in pratica l'algoritmo si comporta in modo efficiente. Tuttavia, l'implementazione classica del Simplesso è intrinsecamente sequenziale, limitando la capacità di sfruttare l'hardware di calcolo moderno.

L'obiettivo di questo progetto è l'implementazione di un solver Simplesso su architettura GPU mediante CUDA, analizzando le sfide legate alla parallelizzazione di un algoritmo iterativo con dipendenze sequenziali. Vengono di seguito presentati i fondamenti teorici, le problematiche di parallelizzazione, l'architettura software sviluppata e i risultati sperimentali ottenuti su problemi di benchmark Netlib.

2 Fondamenti Teorici

2.1 Il Metodo del Simplesso in Formato Tableau

Il metodo del Simplesso opera iterativamente spostandosi da una soluzione di base ammissibile (BFS) a un'altra, migliorando progressivamente il valore della funzione obiettivo fino a raggiungere l'ottimalità. Ogni iterazione del Simplesso comprende tre fasi fondamentali:

Pricing: identificazione della variabile entrante selezionando la colonna con costo ridotto negativo più piccolo (per problemi di minimizzazione). Se tutti i costi ridotti sono non negativi, la soluzione corrente è ottima.

Rapporti minimi: determinazione della variabile uscente calcolando il minimo rapporto tra i termini noti e i coefficienti positivi della colonna pivot.

Pivot: aggiornamento del tableau mediante operazioni elementari di riga per trasformare la colonna pivot in un vettore di unità con elemento 1 nella posizione pivot.

2.2 Metodo delle Due Fasi

Quando non è immediatamente disponibile una BFS iniziale, si ricorre al metodo delle due fasi. La Fase 1 introduce variabili artificiali e minimizza la loro somma: se il valore ottimo è zero, le variabili artificiali possono essere eliminate e si procede alla Fase 2 con l'obiettivo originale. Se il valore ottimo è positivo, il problema originale è inammissibile.

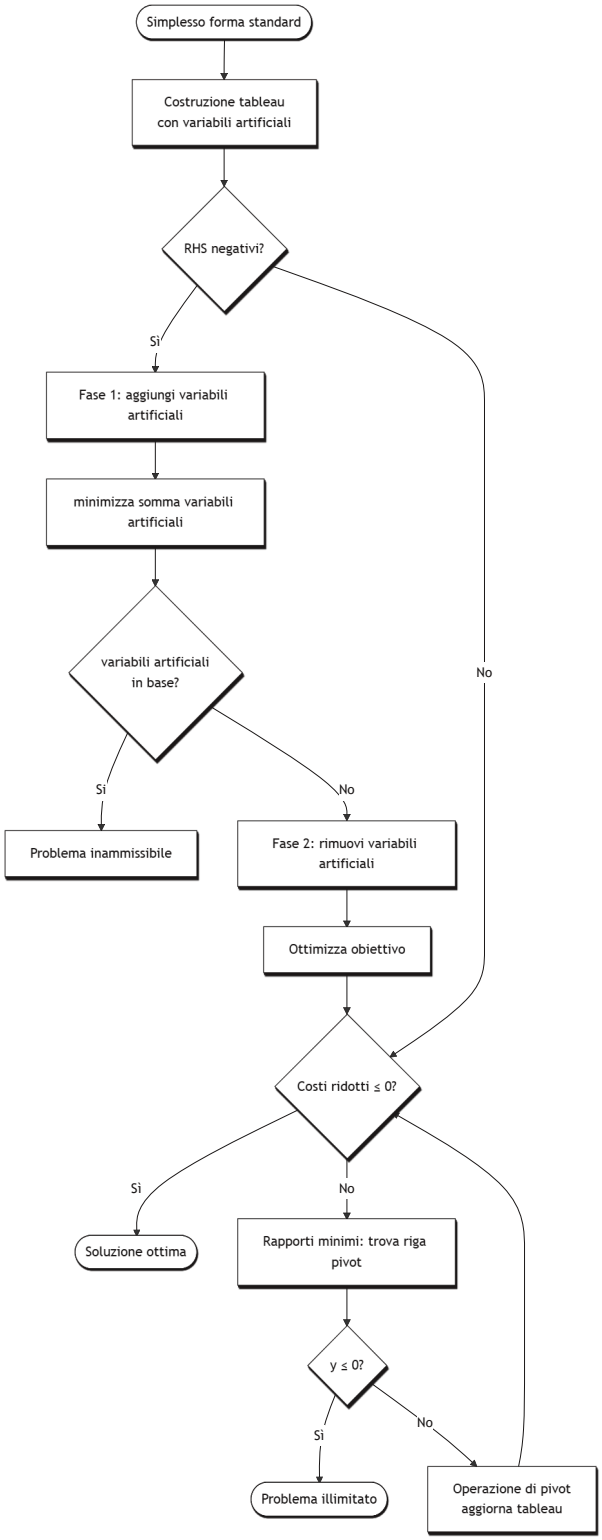

Figura 1: Diagramma di flusso dell'algoritmo Simplesso con metodo delle due fasi.

3 Analisi delle Sfide

3.1 Sfide di Parallelizzazione

La parallelizzazione del metodo del Simplesso presenta sfide significative dovute alla natura intrinsecamente sequenziale dell'algoritmo:

Dipendenze iterazionali: ogni iterazione dipende completamente dal risultato della precedente. Il tableau aggiornato dopo un'operazione di pivot è l'input per la successiva ricerca della colonna pivot, impedendo il parallelismo a livello di iterazioni.

Branch divergence: i rapporti minimi richiedono confronti condizionali che causano divergenza nei warp CUDA. I thread dello stesso warp che seguono percorsi di esecuzione diversi devono essere serializzati, riducendo l'efficienza.

Accesso irregolare alla memoria: l'operazione di pivot accede a righe e colonne specifiche del tableau, creando pattern di accesso non coalesced che limitano la bandwidth effettiva della memoria GPU.

3.2 Stabilità Numerica

La stabilità numerica rappresenta un'altra sfida critica. Le operazioni in floating-point accumulano errori di arrotondamento che possono compromettere la correttezza della soluzione:

Errore di arrotondamento: l'aggiornamento ripetuto del tableau amplifica gli errori numerici, specialmente su problemi mal condizionati.

Scelta progettuale: è stato adottato il tipo `double` per tutte le computazioni numeriche, accettando il costo in memoria e bandwidth in cambio di maggiore precisione. Sui problemi Netlib testati, l'errore relativo sulla funzione obiettivo rimane inferiore allo 0.02%.

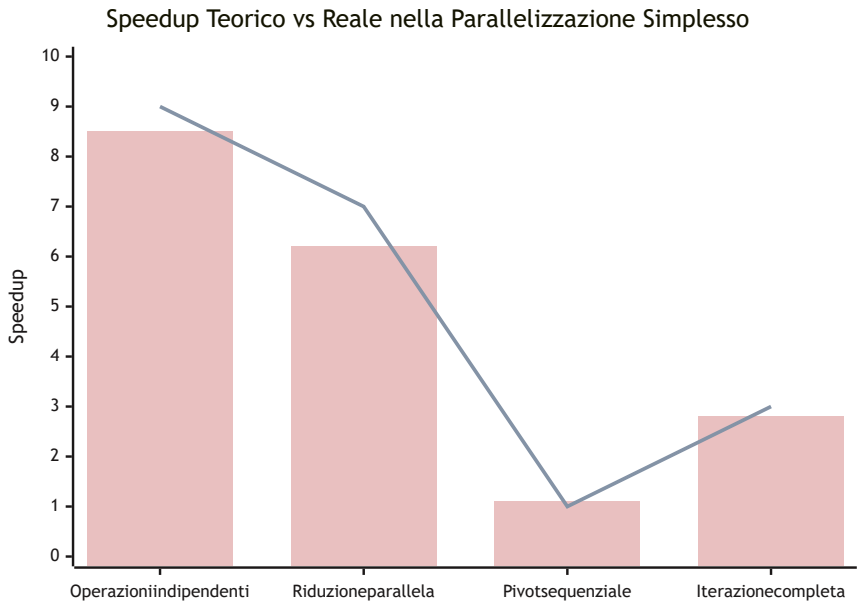

Figura 2: Confronto tra speedup teorico ideale (linea) e speedup misurato (barre) per diverse operazioni dell'algoritmo. Il pivot sequenziale rappresenta il collo di bottiglia principale.

4 Progettazione CUDA

4.1 Architettura del Sistema

Il sistema è organizzato in moduli distinti che separano chiaramente le responsabilità:

- **Parser:** lettura e interpretazione dei file in formato MPS, con supporto per bounds, ranges e variabili intere
- **Solver CUDA:** orchestrazione dei kernel e gestione della memoria device
- **Kernels:** operazioni parallele su GPU (pricing, rapporti minimi, pivot)
- **I/O:** gestione input/output e formattazione risultati

Il flusso dei dati segue un percorso ben definito: il file MPS viene parsato in strutture host, il problema viene preprocessato, il tableau viene costruito e trasferito sulla GPU, l'algoritmo viene eseguito interamente sul device, infine i risultati vengono trasferiti in host per la presentazione.

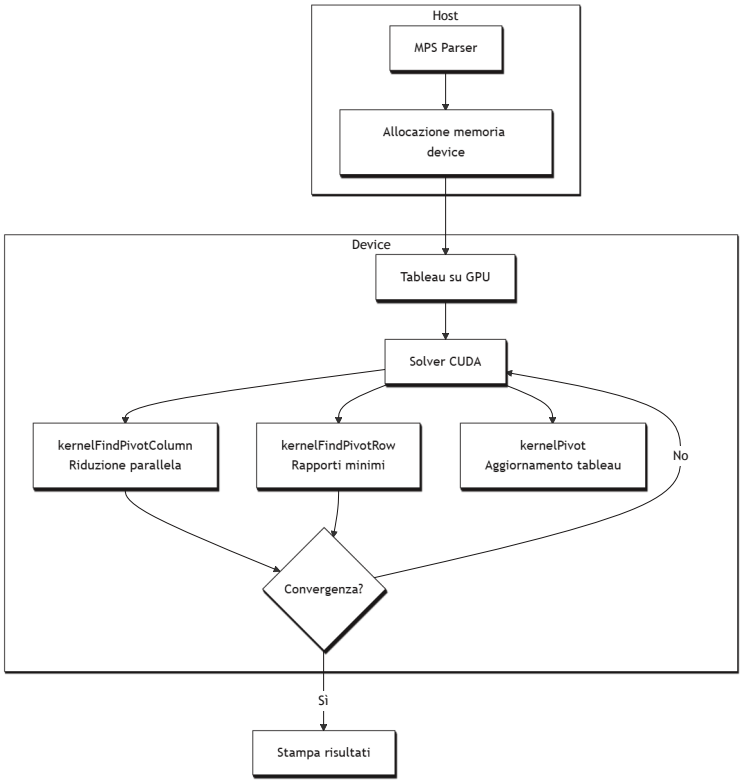

Figura 3: Architettura software e flusso dei dati tra host e device.

4.2 Struttura Dati

La struttura `LPProblem` mantiene la rappresentazione compatta del problema con matrice dei vincoli in formato denso, vettori dei termini noti e dei coefficienti obiettivo, bounds inferiori e superiori, e flag per variabili intere.
Il tableau viene memorizzato in memoria contigua per garantire accessi ottimizzati durante le operazioni di pivot.

4.3 Kernel CUDA

I kernel sono progettati per massimizzare il parallelismo disponibile:

kernelFindPivotColumn: implementa una riduzione parallela utilizzando warp shuffle instructions per trovare il costo ridotto minimo. Ogni warp elabora un sottoinsieme di colonne, quindi viene eseguita una riduzione inter-warp mediante *shared memory*.

kernelFindPivotRow: esegue il rapporto minimo in parallelo con un thread per riga. I candidati validi vengono accumulati in *shared memory* e su di essi viene eseguita una riduzione per trovare il rapporto minimo.

kernelPivot: l'operazione di pivot è parallelizzata assegnando un thread a ogni elemento del tableau. Il pivoting viene eseguito in-place minimizzando l'uso di memoria aggiuntiva.

5 Risultati Sperimentali

5.1 Dataset e Metodologia

I risultati di questa sezione sono stati ottenuti con benchmark reali eseguiti nella macchina Linux messa a disposizione dell'ateneo. Inoltre, sono stati eseguiti dei confronti con uno dei solver più conosciuti in letteratura: **HiGHS**.

Ambiente di test:

- CPU: Intel Xeon E5-2603 v4 @ 1.70GHz (12 core logici totali nel nodo)
- GPU: 3 x NVIDIA GeForce GTX 1070, 8 GB, compute capability 6.1

Dataset usati:

- Benchmark CUDA-only: tutti i file `.dat` in `data/Dati-LP`
- Benchmark CPU vs GPU: file `.mps` in `data/netlib`

5.2 Prestazioni CUDA su Dati-LP

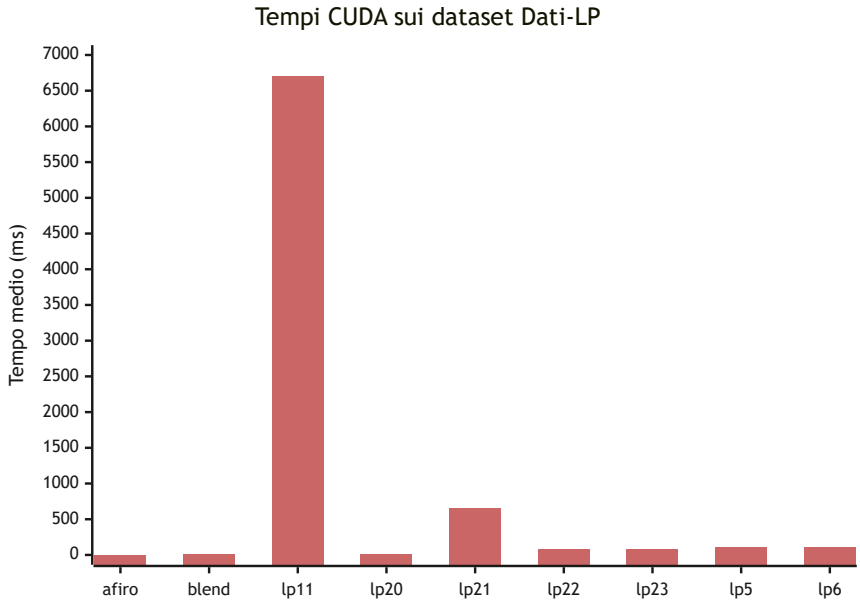

Figura 4: Tempi medi del solver CUDA sui dataset `data/Dati-LP`.

5.3 Confronto Solver CUDA e HiGHS su Netlib

È stato eseguito un confronto sui dataset Netlib in formato MPS, prendendo il famoso solver HiGHS su CPU come riferimento, nella sua implementazione di libreria.

Si osserva che, su questo set, la versione CUDA resta generalmente piu lenta della CPU: nella maggior parte dei casi il fattore di rallentamento e compreso tra circa 1.5x e 3x. Questo comportamento e coerente con la natura del Simplesso tableau-based: le riduzioni parallele (pricing e ratio test) beneficiano della GPU, ma il costo complessivo e ancora dominato da operazioni con forte dipendenza iterativa e accessi memoria non ideali durante i pivot.

In sintesi, il confronto non indica ancora un vantaggio prestazionale assoluto della GPU su questi Netlib, ma evidenzia chiaramente dove intervenire: riduzione del traffico globale, migliore riuso dati in shared memory e strategie di pivot piu cache-friendly.

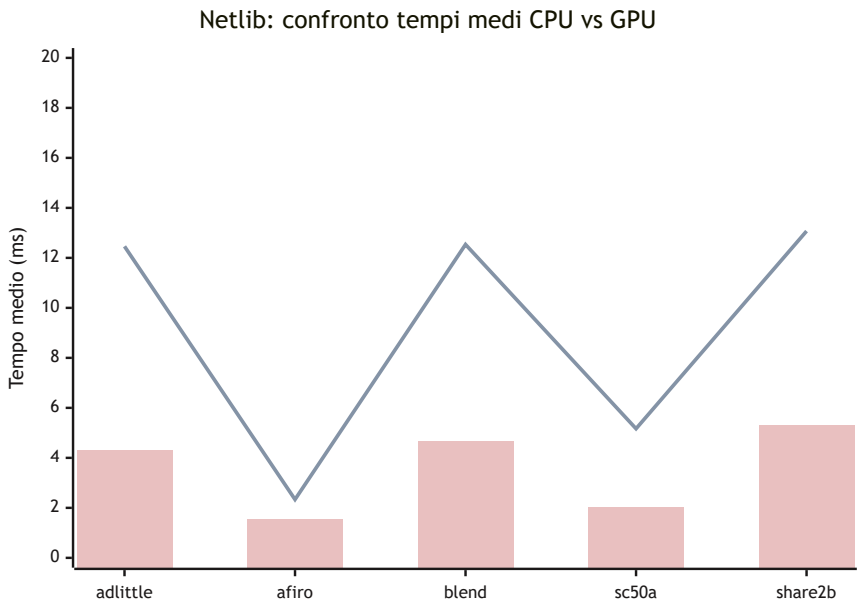

Figura 5: Barre = HiGHS, linea = CUDA.

5.4 Errore Numerico su Netlib

L'errore relativo sulla funzione obiettivo è risultato estremamente contenuto.

In dettaglio, tutti i valori restano nell'ordine di 10^-9 o inferiori. Questo livello di scostamento e compatibile con le differenze attese tra implementazioni floating-point diverse (ordine delle operazioni, riduzioni parallele, gestione dei pivot quasi degeneri) e conferma che l'uso di `double` mantiene la soluzione numericamente affidabile.

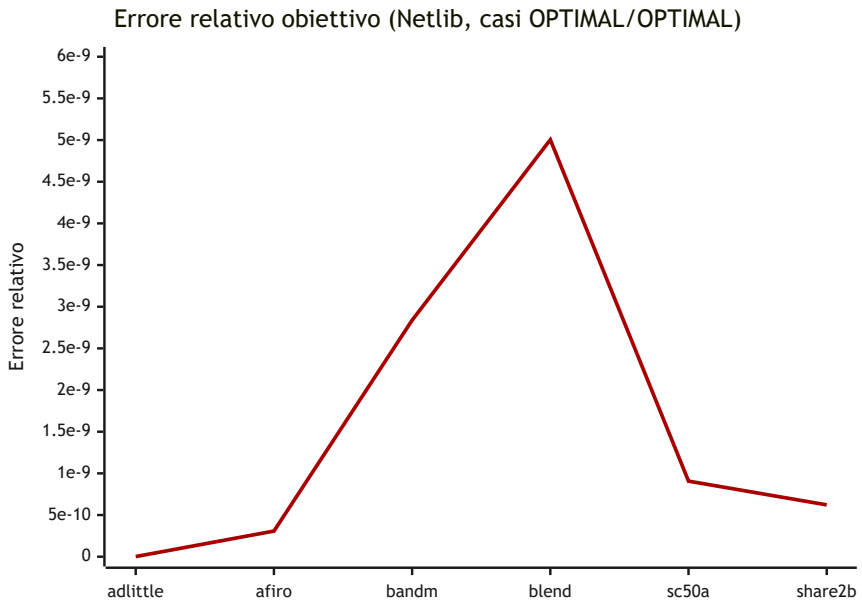

Figura 6: Errore relativo rispetto a HiGHS.

6 Conclusioni

Questo progetto ha dimostrato la fattibilità dell'implementazione del metodo del Simplesso su architettura GPU, evidenziando allo stesso tempo le sfide intrinseche nella parallelizzazione di algoritmi iterativi con forti dipendenze sequenziali.

I benchmark reali su questa macchina mostrano accuratezza numerica elevata nei casi in cui CUDA e HiGHS ma anche prestazioni del solver custom complessivamente inferiori a quello commerciale sul set Netlib testato. Le ottimizzazioni implementate (memoria coalesced e riduzioni parallele) costituiscono comunque una base solida per ulteriori miglioramenti di robustezza e performance.

Le principali lezioni apprese riguardano l'importanza della gestione memoria in CUDA e il trade-off tra precisione e prestazioni. L'uso di double precision, sebbene più costoso in termini di bandwidth, si è rivelato necessario per garantire la stabilità numerica.